Paroles : Veille Fetch vs Axios

Moaad et Adam · YouCode · document privé, à ne pas projeter

On ne lit pas les slides : ce texte est pour vous seuls. Dites-le avec vos mots, ce sont des repères, pas un texte à réciter. Les passages en *italique* sont des gestes à faire (montrer quelque chose à l'écran). Durée totale prévue : environ 28 minutes.

Slide 1 · Titre (1 min)

Moaad : Bonjour à tous. Moi, c'est Moaad, et voici Adam. Aujourd'hui, on va vous parler de deux outils pour récupérer des données depuis un serveur en JavaScript : Fetch et Axios. On va d'abord voir comment ils s'écrivent, ensuite comment ils gèrent les erreurs, puis on vous montre un tableau qui compare tout ça, et on finit par une petite démo dans la console.

Adam : Et à la fin, on répondra aussi aux trois questions de réflexion.

Moaad : On commence par présenter les deux outils.

Slide 2 · Les deux outils (3 min)

Moaad : Avant de comparer, rappelons ce qu'est une requête GET. C'est simplement quand notre page demande une information à un serveur, par exemple une liste d'articles, sans rien modifier là-bas.

Pour faire ça en JavaScript, il y a deux façons très courantes. La première, c'est Fetch. Il est déjà intégré au navigateur, donc on n'a rien à installer, on peut l'utiliser tout de suite. *(montrer la carte Fetch)*

La deuxième, c'est Axios. Lui, c'est une bibliothèque externe : il faut l'ajouter à notre projet, soit avec npm install axios, soit avec un lien CDN dans le fichier HTML. Un CDN, c'est un lien qui charge la bibliothèque depuis internet. *(montrer la carte Axios)*

Les deux servent à la même chose, mais ils ne s'écrivent pas pareil et ne réagissent pas pareil. C'est ce qu'on va voir.

Slide 3 · Requête GET côte à côte (5 min)

Moaad : Voici le même appel écrit avec les deux. *(montrer la colonne de gauche)* À gauche, Fetch. On écrit fetch avec l'URL, et ça nous renvoie une promesse. Le premier then reçoit la réponse brute, mais ce ne sont pas encore nos données. Il faut appeler r.json() pour les convertir, et c'est seulement dans le deuxième then qu'on peut les afficher.

(montrer la colonne de droite) À droite, Axios. On écrit axios.get avec l'URL, et dans le then, les données sont déjà converties. On les lit directement dans r.data, donc un seul then suffit.

La grosse différence de syntaxe, c'est ça : r.json() avec Fetch, r.data avec Axios. Dans les deux cas, on termine avec un catch pour attraper les problèmes. Adam va vous expliquer justement ce qui se passe quand ça se passe mal.

Slide 4 · Gestion des erreurs (5 min)

Adam : Merci Moaad. Ici, on utilise async await avec try catch, qui est une autre façon d'écrire la même chose.

(montrer la ligne if (!r.ok) à gauche) Regardez cette ligne dans Fetch. Pourquoi elle est là ? Parce que Fetch ne considère pas une erreur 404 ou 500 comme une vraie erreur. Il ne rejette la promesse que si le réseau est coupé, par exemple. Donc si la page n'existe pas, le code continue comme si de rien n'était. C'est à nous de tester r.ok et de lancer l'erreur nous-mêmes.

(montrer la colonne de droite) Avec Axios, pas de test à écrire. Dès que le statut n'est pas entre 200 et 299, Axios envoie directement dans le catch.

En résumé : Axios gère ça tout seul, Fetch demande une ligne de plus.

Slide 5 · Tableau comparatif (4 min)

Adam : Ce tableau résume tout ce qu'on vient de voir. *(descendre ligne par ligne)*

Installation : Fetch est natif, Axios est à ajouter. Lecture du JSON : r.json() d'un côté, r.data de l'autre. Erreurs 404 et 500 : avec Fetch, on teste r.ok, avec Axios, elles arrivent dans le catch. En-têtes : les deux acceptent l'option headers, mais Axios permet de les fixer une fois avec axios.create(). Timeout : Fetch passe par AbortController, Axios a une simple option timeout. Poids : Fetch n'ajoute rien, Axios est une dépendance en plus.

Il n'y a pas de gagnant absolu, ça dépend du besoin. Moaad va maintenant répondre aux questions de réflexion.

Slide 6 · Questions de réflexion (5 min)

Moaad : Première question : pourquoi choisir Axios quand on a des en-têtes complexes ? Les en-têtes, ce sont des informations qu'on envoie avec la requête, comme un token d'autorisation ou le type de contenu. Avec Axios, on peut utiliser axios.create() pour les définir une seule fois, et toutes les requêtes les reprennent. On peut aussi utiliser des intercepteurs pour ajouter un token automatiquement à chaque requête. Avec Fetch, on doit réécrire les en-têtes à chaque appel, ou créer sa propre fonction pour le faire.

Adam : Deuxième question : dans quel cas Fetch est plus avantageux ? Pour une requête simple ou un petit projet : rien à installer, aucune dépendance, il est déjà dans le navigateur.

Troisième question : que se passe-t-il si le serveur renvoie autre chose que du JSON ? Avec Fetch, r.json() déclenche une erreur, donc il faut lire la réponse avec r.text(). Avec Axios, si la conversion échoue, le texte reste tel quel dans r.data, et il n'y a pas d'erreur.

Slide 7 · Démo et conclusion (5 min)

Moaad : On lance la démo. *(ouvrir la console du navigateur, lancer Fetch puis Axios sur la même URL de test)*

Adam : On compare ce qui s'affiche : les deux donnent le même objet, mais avec Fetch on a dû convertir avec r.json(), alors qu'avec Axios c'est déjà fait dans r.data.

Moaad : Pour conclure : Fetch pour une requête simple, Axios quand le projet grandit et qu'on répète les en-têtes, les erreurs HTTP ou le timeout.

Adam : Merci, on est prêts pour vos questions.

Avant de passer

- Parlez lentement et pointez le code à l'écran pendant que vous l'expliquez.
- Moaad garde la démo prête dans un onglet, Adam surveille le temps.
- Si vous êtes en avance, prenez plus de temps sur les erreurs (slide 4), c'est la partie qui se retient le mieux.